

HSBC Interview Preparation

8-Session Crash Guide for Linux, Ansible, Python, REST API, CI/CD, SOX, Kubernetes, Monitoring, SQL, and Hadoop

Prepared for Shekar - focused on BFSI production support, automation, compliance, and real-time issue explanation.

JD Focus Area	How to position your experience
Linux, Ansible, Python, DevOps Tooling	Strong production support plus automation background.
REST APIs and CI/CD	Tool integration, quality gates, change approval, deployment, validation, notifications.
SOX and compliance	Approved change, audit trail, rollback plan, evidence, controlled production operations.
Kubernetes, Grafana, Prometheus	Troubleshooting, monitoring, alerts, post-deployment validation.
SQL, Hadoop basics	Operational awareness: connectivity, sessions, logs, storage, YARN/HDFS checks.

Preparation Order

1. Linux + patching
2. Ansible + Tower
3. Python + REST API
4. CI/CD + GitHub
5. SOX compliance
6. Kubernetes + monitoring
7. SQL + Hadoop basics
8. Mock interview

Use this guide as a fast revision document. For each topic, focus on three things: what it is, where it is used in production, and one real issue with root cause and fix.

Session 1: Linux Production Troubleshooting + Patching

What to know

Linux hosts applications, databases, middleware, scripts, monitoring agents, batch jobs, and automation agents. In interviews, explain Linux using production troubleshooting flow.

```
uptime
top
free -m
df -h
du -sh *
ps -ef
systemctl status <service>
journalctl -xe
tail -f /var/log/messages
ss -tulnp
curl http://localhost:8080/health
```

Real-time issue

Application became slow after monthly patching. The team checked CPU, memory, disk, service status, and application logs.

```
top
free -m
df -h
systemctl status payment-api
journalctl -u payment-api
```

- Finding: memory usage was high and the application started swapping.
- Root cause: JVM heap increased after deployment, but server memory was not enough.
- Fix: restarted service as immediate workaround, increased memory after approval, and asked application team to tune JVM heap.

Patching workflow

```
Change ticket approved
↓
Pre-check
↓
Backup/snapshot confirmation
↓
Patch non-prod
↓
Patch prod in batches
↓
Reboot if required
↓
Post-check
↓
```

```
Attach evidence
↓
Close change

yum update -y
dnf update -y
needs-restarting -r
uname -r
rpm -qa --last | head
```

Interview answer: I handle Linux production troubleshooting by checking CPU, memory, disk, services, logs, ports, and application health. For patching, I follow approved change process with pre-checks, batch execution, reboot validation, post-checks, and evidence attachment.

Session 2: Ansible + Ansible Tower

Ansible basics

Ansible automates repeatable server tasks such as package installation, service restart, config copy, patching, pre-checks, post-checks, and report collection.

```
---
- name: Restart application
  hosts: app_servers
  become: yes

tasks:
- name: Restart payment service
  service:
    name: payment-api
    state: restarted
```

Important Ansible concepts

Concept	Meaning
Inventory	Server list
Playbook	YAML automation file
Module	Built-in task like yum, copy, service
Role	Reusable playbook structure
Variable	Dynamic value
Handler	Runs only when notified
Register	Stores command output
When	Conditional execution
Vault	Encrypts secrets
Serial	Runs in batches

Ansible Tower / Automation Controller

Tower provides RBAC, audit logs, job history, credentials, scheduling, workflow templates, survey forms, notifications, and REST API access.

Real-time Tower patching workflow

```
Pre-check job
↓
Patch job
↓
Reboot job
↓
Post-check job
↓
Report generation
```

Issue: one server failed after reboot. Tower showed wait_for_connection failed. Root cause was /var filesystem became 100% full due to yum cache.

```
yum clean all
systemctl restart sshd
```

Prevention: added disk check in pre-check playbook.

Interview answer: I use Ansible for repeatable infrastructure automation and Tower for controlled enterprise execution. Tower helps with RBAC, audit, scheduling, workflow, and job history, which is very important in BFSI environments.

Session 3: Python + REST API

Python use in DevOps

- REST API integration
- Health checks
- Report generation
- Log parsing
- CSV/Excel reports
- ServiceNow updates
- Jenkins/Ansible Tower automation
- Prometheus data collection

```
import requests

url = "https://payment.company.com/health"
response = requests.get(url, timeout=10)

if response.status_code == 200:
    print("Application is healthy")
else:
    print("Application is unhealthy")
exit(1)
```

REST API basics

REST API allows tools to communicate over HTTP. In DevOps, REST APIs connect ServiceNow, Jenkins, GitHub, SonarQube, Ansible Tower, ArgoCD, Prometheus, and notification tools.

Method	Usage
GET	Read data
POST	Create or trigger action
PATCH	Partial update
PUT	Replace full object
DELETE	Delete object

Code	Meaning
200	Success
201	Created
400	Bad request
401	Unauthorized
403	Forbidden
404	Not found
500	Server error

Interview answer: I use Python for custom DevOps automation, especially where REST API integration is required. For simple API calls, curl is enough. For advanced logic like JSON parsing, retries, multiple endpoints, and reporting, Python requests module is better.

Session 4: CI/CD + GitHub

CI/CD flow

```

Code commit
↓
Build
↓
Unit test
↓
Code scan
↓
Artifact/image build
↓
Image scan
↓
Deploy
↓
Post-deployment validation
↓
Notification

```

JD angle

The JD expects build, scan, test, deployment automation, DevSecOps principles, GitHub branch management, code review, and merge strategy awareness.

GitHub concepts

Concept	Meaning
Branch	Separate line of work
Pull Request	Request to merge code
Code Review	Peer review before merge
Merge	Combine code
Rebase	Replay commits on latest base
Squash merge	Combine commits into one

Tag	Release marker
Git revert	Safe rollback

Real-time CI/CD flow

```

Developer raises PR
↓
Code review completed
↓
JUnit test runs
↓
SonarQube scan passes
↓
Docker image builds
↓
Image pushed to registry
↓
ServiceNow change approval checked
↓
Deployment triggered
↓
Prometheus/Grafana validation
↓
Notification sent

```

Interview answer: I have worked with CI/CD from build, test, scan, deploy, and validation perspective. In a controlled environment, pipeline should include code review, quality gates, security scanning, approval validation, deployment, monitoring validation, and audit evidence.

Session 5: SOX, Compliance, and Production Change Control

SOX means Sarbanes-Oxley compliance. In BFSI, production changes must be controlled, approved, traceable, and auditable.

Production change must have

- Change ticket
- Business justification
- Risk assessment
- Implementation plan
- Rollback plan
- Validation plan
- Approval
- Execution evidence
- Post-change validation
- Closure comments

Real-time patching evidence

- Approved ServiceNow ticket
- Server list
- Pre-check output
- Patch job output

- Reboot status
- Post-check output
- Application validation
- Grafana screenshot
- Rollback plan

What not to do

Wrong:
`kubectl edit deployment payment-api -n prod`

Better:
 Update Git
 Raise PR
 Get approval
 Merge
 ArgoCD sync
 Attach evidence

Interview answer: In BFSI, we follow strict change management. Every production change should have approval, implementation plan, rollback plan, validation evidence, and audit trail. Tools like GitHub, Jenkins, Ansible Tower, ArgoCD, and ServiceNow help maintain traceability.

Session 6: Kubernetes + Prometheus + Grafana

Kubernetes troubleshooting commands

```
kubectl get pods -n <ns>
kubectl describe pod <pod> -n <ns>
kubectl logs <pod> -n <ns>
kubectl logs <pod> -n <ns> --previous
kubectl get events -n <ns> --sort-by=.lastTimestamp
kubectl get svc -n <ns>
kubectl get endpoints <svc> -n <ns>
kubectl rollout status deployment <deploy> -n <ns>
```

Common Kubernetes issues

Issue	Cause
CrashLoopBackOff	Application crash or config issue
ImagePullBackOff	Wrong image or registry auth issue
Pending	No resources or PVC issue
Service endpoint empty	Selector mismatch or pods not ready
503 from ingress	No healthy backend
ConfigMap issue	Wrong config value
Secret issue	Missing or wrong password

Real-time issue: frontend got 503

```
kubectl get svc -n payments
kubectl get endpoints payment-api -n payments
kubectl get pods -n payments --show-labels
```

Finding: Service endpoint was empty. Root cause: Service selector did not match pod label. Fix: corrected selector in Git and synced through ArgoCD.

Prometheus and Grafana

Prometheus collects metrics. Grafana visualizes dashboards. Useful metrics include pod restart count, CPU, memory, HTTP 5xx, latency, node status, and disk usage.

```
increase(kube_pod_container_status_restarts_total[15m])

sum(rate(http_requests_total{status=~"5.."}[5m]))
```

A good Grafana dashboard should show availability, request count, error rate, P95/P99 latency, pod restarts, CPU/memory, and database connection usage.

Interview answer: I troubleshoot Kubernetes by checking pods, events, logs, deployment, service, endpoints, ingress, ConfigMap, Secret, and metrics. Prometheus gives metrics and Grafana gives dashboard visibility.

Session 7: SQL + RDBMS + Hadoop Basics

SQL / RDBMS basics

The JD mentions Postgres and Oracle. Focus on operational troubleshooting, not DBA-level depth.

- Connection issue
- Authentication issue
- Slow query
- High sessions
- Locks
- Storage/tablespace full
- Listener issue
- Backup/restore awareness

```
select count(*) from table_name;
select * from table_name where id = 100;

-- Postgres
select count(*) from pg_stat_activity;
select * from pg_stat_activity where state='active';
```

Oracle concepts to know: tablespace, listener, sessions, locks, alert log, archive logs.

Real-time DB issue

Application returned HTTP 500. Logs showed connection pool exhausted. Checks included DB connectivity, credentials, active sessions, connection pool, slow queries, and locks. Root cause was application connection leak. Fix was restart as temporary relief, tune pool, and have application team fix leak.

Hadoop basics

Component	Meaning
HDFS	Distributed file system
NameNode	Stores metadata
DataNode	Stores data blocks
YARN	Resource manager
ResourceManager	YARN master
NodeManager	Runs jobs on workers

MapReduce	Batch processing
Hive	SQL on Hadoop
Spark	Fast processing engine

```
hdfs dfs -ls /
hdfs dfs -du -h /path
hdfs dfsadmin -report
yarn application -list
yarn logs -applicationId <id>
```

Interview answer: I have basic database and Hadoop operational awareness. For databases, I check connectivity, sessions, locks, slow queries, and storage. For Hadoop, I understand HDFS, NameNode, DataNode, YARN, and how to check HDFS health and YARN application logs.

Session 8: Final HSBC Mock Interview Answers

1. Tell me about yourself

I have around 15+ years of IT experience, mainly in Linux infrastructure, production support, automation, and cloud operations. I have worked in BFSI environments, handling incidents, changes, patching, DR activities, and operational automation. My strengths are Linux, Ansible, Ansible Tower, production troubleshooting, ServiceNow change process, and automation. I also have exposure to Python, REST API integration, CI/CD, Kubernetes, Prometheus, Grafana, and database troubleshooting.

2. How do you use Ansible?

I use Ansible to automate repeatable infrastructure tasks like patching, service restart, configuration deployment, pre-checks, post-checks, and report collection. I use playbooks, roles, variables, handlers, register, when conditions, and serial execution for controlled rollout.

3. What is Ansible Tower?

Ansible Tower is an enterprise automation platform for running Ansible playbooks with RBAC, credentials, scheduling, workflow, job history, notifications, and audit logs. In BFSI, Tower is useful because it provides controlled and auditable automation.

4. Explain your patching workflow

I start with approved change ticket, then run pre-checks, validate backup/snapshot, patch in batches, reboot if required, run post-checks, validate application health, attach evidence, and close the change.

5. How do you use Python in DevOps?

I use Python for custom automation, REST API integration, health checks, JSON parsing, report generation, and integration with tools like ServiceNow, Jenkins, Prometheus, and Ansible Tower.

6. Where do you use REST API in CI/CD?

REST APIs are used to integrate tools. Jenkins can call SonarQube API for quality gate, ServiceNow API for change approval, Ansible Tower API to trigger automation, ArgoCD API to sync Kubernetes apps, Prometheus API to validate metrics, and Teams/Slack API for notifications.

7. Explain CI/CD pipeline

A CI/CD pipeline automates code build, test, scan, package, deploy, and validation. In DevSecOps, we include unit tests, code scan, image scan, approval gate, deployment, monitoring validation, and audit evidence.

8. How do you troubleshoot Kubernetes pod issue?

I check pod status, describe output, events, logs, previous logs, resource usage, ConfigMap, Secret, readiness/liveness probes, service endpoints, and deployment rollout.

9. How do you troubleshoot service endpoint empty issue?

I check service selector, pod labels, pod readiness, namespace, and EndpointSlice. Most common cause is selector mismatch or pods not ready.

10. What is Prometheus and Grafana?

Prometheus collects metrics and Grafana visualizes them. I use them to monitor pod restarts, CPU, memory, latency, HTTP errors, availability, and infrastructure health.

11. How do you handle production compliance?

I follow change approval, rollback plan, implementation plan, validation evidence, segregation of duties, and proper documentation. This supports SOX and audit requirements.

12. What is your strength for this role?

My strength is production support with automation. I understand Linux deeply, have Ansible/Tower experience, follow change/compliance process, and can support DevOps tooling, CI/CD, Python, REST API integration, Kubernetes, and monitoring.

Final 2-Minute Interview Answer

Interview answer: I come from a strong Linux production support and BFSI background. I have handled incidents, changes, patching, DR activities, and automation using Ansible and Ansible Tower. I understand controlled production environments where approvals, rollback plans, validation, and audit evidence are very important. I use Ansible for server automation and Python/REST APIs for tool integration with ServiceNow, Jenkins, Ansible Tower, Prometheus, and other DevOps tools. I also have working knowledge of CI/CD, GitHub, Kubernetes troubleshooting, Prometheus, Grafana, SQL basics, and Hadoop operational concepts. My focus is stable production operations, automation, compliance, and reducing manual effort.